AI-Powered Cyber Incident Monitoring Tool Documentation

Release 1.0.0

Daniel Vetrila

April 7, 2025

Contents

CHAPTER

ONE

1. INTRODUCTION

1 1.1. Project Overview

This document provides comprehensive documentation for the AI-Powered Cyber Incident Monitoring Tool, a sophisticated system designed to automate the collection, analysis, and reporting of cybersecurity threats. The tool leverages cutting-edge technologies including Google's Gemini AI for advanced analysis, the ThreatFox API for IOC (Indicator of Compromise) enrichment, and mapping against the MITRE ATT&CK® framework to provide context to identified threats. The primary goal is to enhance the efficiency and accuracy of threat detection and response, empowering cybersecurity professionals to proactively defend against evolving cyber threats by providing timely, actionable intelligence through both a web interface and a command-line interface (CLI).

2 1.2. Background and Motivation

The cybersecurity landscape is characterized by an ever-increasing volume and sophistication of threats. Traditional manual methods of threat monitoring and analysis are often insufficient to keep pace with the speed and stealth of modern attacks. Security teams can be overwhelmed by the sheer amount of data, making it challenging to identify genuine threats in a timely manner and respond effectively. This project is motivated by the critical need to:

- **Automate** the labor-intensive process of threat data collection and initial IOC identification.
- Provide **real-time analysis** of potential threats to reduce detection and response times.
- **Integrate multiple threat intelligence sources** to build a more comprehensive understanding of IOCs.
- Generate **actionable insights** that are directly relevant to security operations teams.
- **Standardize threat reporting** using established frameworks like MITRE ATT&CK® for better communication and understanding of adversarial tactics, techniques, and procedures (TTPs).
- **Reduce response fatigue** by helping to prioritize threats, allowing teams to focus on the most critical incidents.

This tool aims to address these challenges by providing an intelligent, automated solution that assists cybersecurity professionals in navigating the complex threat landscape.

3 1.3. Project Objectives

The primary objectives for the AI-Powered Cyber Incident Monitoring Tool are:

1. **Automated Threat Data Collection**: Continuously gather articles and threat data from predefined web sources (NewsNow, ThreatPost RSS).

2. **AI-Driven IOC Extraction and Analysis**: Utilize Google's Gemini AI to extract potential IOCs from collected data and perform in-depth analysis of these IOCs and related article content.
3. **IOC Enrichment via ThreatFox**: Integrate with the ThreatFox API to enrich identified IOCs with up-to-date threat intelligence.
4. **MITRE ATT&CK® Framework Mapping**: Map enriched IOC data to the MITRE ATT&CK® framework to provide standardized context on adversary behaviors.
5. **Web Interface for Visualization and Interaction**: Develop a user-friendly web interface (FastAPI, Jinja2) to display monitored articles, IOCs, detailed analysis reports, and allow manual IOC submission and PCAP file analysis, with real-time updates via WebSockets.
6. **Command-Line Interface (CLI)**: Provide a basic CLI for IOC analysis and report generation, catering to automation and scripting needs.
7. **Automated HTML Report Generation**: Generate comprehensive HTML reports for analyzed IOCs.
8. **System Reliability and Maintainability**: Ensure the system is designed modularly, handles errors gracefully, and manages configurations securely.

4 1.4. Scope of Work

4.1 1.4.1. In Scope

The project encompasses the following functionalities and features, as detailed in the project specification:

- **Data Collection**: Automated collection from NewsNow (Cyber Attacks section) and ThreatPost RSS feed.
- **IOC Extraction**: Using regular expressions and Gemini AI from collected articles.
- **Gemini AI Integration**: For IOC extraction, IOC analysis (generating threat level, impact, recommendations), and full article content summarization.
- **ThreatFox API Integration**: For enriching IOCs with details like threat type, malware associations, and confidence levels.
- **MITRE ATT&CK® Mapping**: Utilizing a locally cached MITRE ATT&CK® enterprise dataset (JSON from GitHub) to map IOCs to TTPs.
- **Web Interface (FastAPI)**:
  - Dashboard for recent articles and status.
  - Detailed article view with AI analysis.
  - Manual IOC submission and analysis.
  - Display of HTML reports for IOCs.
  - Real-time updates via WebSockets.
  - PCAP file analysis (using `tshark`) to check for IOC presence.
- **CLI**: Basic functionality for submitting an IOC and receiving an analysis report (FR15).

- **Reporting**: Automated generation of HTML reports for IOC analysis.
- **Caching**: Local caching for MITRE ATT&CK® data with periodic refresh.
- **Configuration**: Secure management of API keys via environment variables (`.env` file).
- **Error Handling**: Graceful handling of API failures and other operational errors.
- **Modularity**: A modular codebase to facilitate maintenance and future enhancements.

4.2 1.4.2. Out of Scope

The initial version of this project will not include:

- Advanced user authentication and role-based access control (RBAC) for the web interface.
- Sophisticated, customizable threat ranking based on detailed organizational profiles.
- Active blocking or remediation capabilities (the tool is for monitoring and analysis).
- Distributed deployment or high-availability clustering.
- Training custom machine learning models for IOC detection (relies on Gemini AI's capabilities).
- Extensive support for a wide array of data sources beyond those initially specified.

CHAPTER

TWO

2. SYSTEM ARCHITECTURE

1 2.1. Architectural Overview

The AI-Powered Cyber Incident Monitoring Tool is designed with a modular architecture to promote separation of concerns, maintainability, and scalability. The system comprises several key interconnected components that handle distinct stages of the threat monitoring and analysis pipeline, from data ingestion to report presentation.

The architecture emphasizes automated workflows, integration with external intelligence services, and providing actionable information to users through multiple interfaces.

2 2.2. Core Components

The system is built around the following primary components:

- **Cyber Article Monitor (`cyber_monitor.py`)**: Responsible for fetching articles from web sources like NewsNow. It handles HTTP requests, parses HTML content, extracts preliminary article details (title, link, timestamp), and performs initial regex-based IOC extraction from titles. It collaborates with the `IOCAnalyzer` for deeper analysis.
- **IOC Analyzer (`ioc_analyzer.py`)**: This is the analytical core of the system. It takes IOCs (or full article content) and:
  - Integrates with **Google's Gemini AI** for detailed textual analysis, summarization, and structured data extraction (e.g., threat level, impact).
  - Connects to the **ThreatFox API** to enrich IOCs with external threat intelligence.
  - Maps IOC characteristics to the **MITRE ATT&CK® framework** using a local cache of MITRE's enterprise data.
  - Generates comprehensive HTML reports detailing the findings.
- **Web Interface & API (`web_monitor.py`)**: A FastAPI-based application that provides:
  - A user interface (built with Jinja2 templates) for real-time monitoring of articles, viewing IOC analyses, submitting IOCs manually, and initiating PCAP scans.
  - WebSocket communication for pushing real-time updates to connected clients.
  - HTTP API endpoints for functionalities like fetching recent ThreatFox IOCs, analyzing submitted IOCs, and analyzing submitted article content.
- **RSS Feed Monitor (`rss_monitor.py`)**: A simpler component specifically designed to fetch and parse data from RSS feeds, such as ThreatPost. It currently acts as a standalone utility for fetching the latest feed items.

- **MITRE ATT&CK® Cache**: A local JSON file (*mitre_attack_cache.json*) storing the MITRE ATT&CK® enterprise dataset. The `IOCAnalyzer` manages this cache, including periodic refreshes, to ensure efficient and up-to-date mapping.

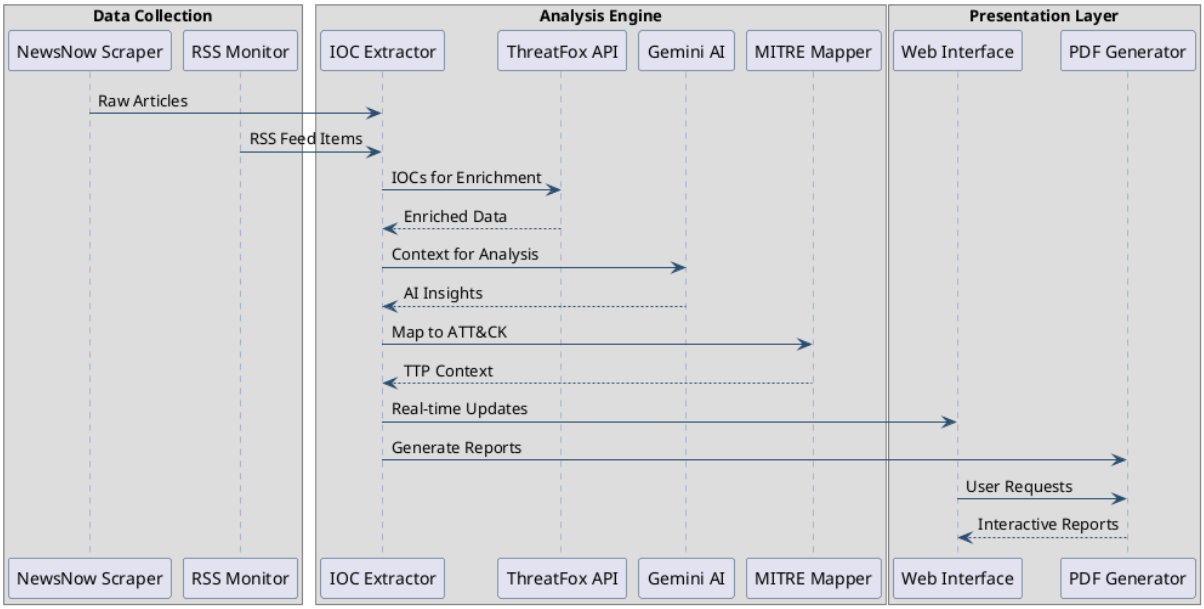

Fig. 1: Data Flow Sequence

3 2.3. Data Flow

The data flow within the system can be summarized as follows:

1. **Data Ingestion**:
   - `CyberMonitor` periodically fetches articles from NewsNow.
   - `RSSMonitor` (conceptually, or if integrated into a main loop) fetches data from ThreatPost RSS.
2. **Initial Processing**:
   - `CyberMonitor` parses HTML, extracts article metadata, and performs regex-based IOC extraction from titles.
3. **IOC/Article Analysis (via `IOCAnalyzer`)**:
   - Extracted IOCs are sent to `IOCAnalyzer`.
   - IOCs are enriched via the **ThreatFox API**.
   - Enriched data and original IOC context are analyzed by **Gemini AI** for threat level, impact, recommendations, etc.
   - IOCs are mapped to **MITRE ATT&CK® TTPs**.
   - Full article content can also be submitted for Gemini AI summarization and analysis.
4. **Reporting**:
   - `IOCAnalyzer` generates an HTML report for each analyzed IOC.
5. **Presentation & Interaction (via `WebMonitor`)**:

- New articles and their analyses are stored and broadcast via WebSockets to the web interface.
- Users can view articles, IOC reports, and submit IOCs or article text for on-demand analysis through the web UI.
- Users can initiate PCAP file scans for specific IOCs.
- The CLI (conceptually, as per FR15) would allow direct IOC submission to `IOCAnalyzer` and report retrieval.

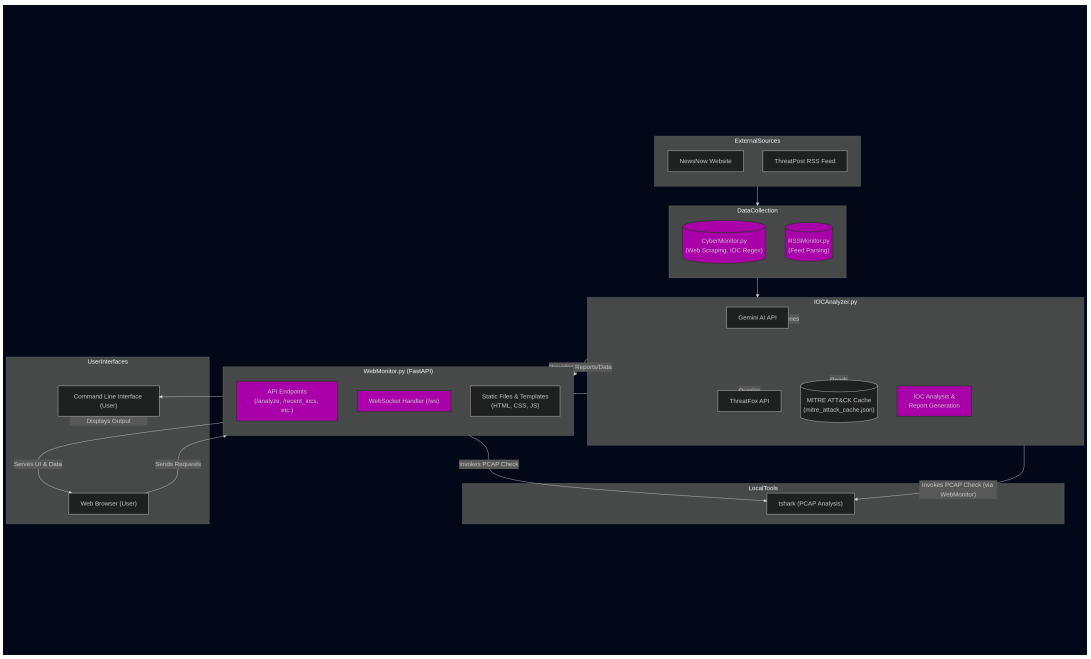

Fig. 2: Diagram illustrating the sequential flow of data from collection through analysis to presentation.

4 2.4. Technologies Used

The project leverages a range of modern technologies:

- **Programming Language**: Python 3.8+
- **Web Framework**: FastAPI (for the web server and API endpoints)
- **Templating Engine**: Jinja2 (for rendering HTML in the web interface)
- **AI Service**: Google Gemini AI (via the *google-generativeai* SDK for IOC analysis and content summarization)
- **Threat Intelligence API**: ThreatFox API (abuse.ch) (for IOC enrichment)
- **Cybersecurity Framework**: MITRE ATT&CK® (enterprise dataset for TTP mapping)
- **Data Fetching/Parsing**: *requests* (HTTP calls), *BeautifulSoup4* (HTML parsing), *feedparser* (RSS parsing)
- **Data Handling**: *stix2* (for MITRE data), *json*
- **CLI Utilities**: *rich* (for potentially enhanced CLI output, though primarily used by IOCAnalyzer for internal formatting that gets converted to HTML), *tqdm* (progress bars for downloads)
- **Environment Management**: *python-dotenv* (for managing API keys via `.env` files)

- **Network Analysis (for PCAP)**: *tshark* (command-line tool, part of Wireshark)
- **Web Server (for FastAPI)**: Uvicorn
- **Version Control**: Git

CHAPTER

THREE

3. IMPLEMENTATION DETAILS

This section delves into the specifics of each core Python module within the project.

1 3.1. Cyber Article Monitor (*cyber_monitor.py*)

The `CyberMonitor` class is responsible for automated fetching and initial processing of articles from web sources, primarily NewsNow.

- **Initialization (`__init__`)**:
  - Instantiates an `IOCAnalyzer` object for subsequent analysis tasks.
  - Sets the target `article_url` (NewsNow Cyber Attacks page).
  - Configures HTTP `headers` to mimic a browser.
  - Initializes an empty `seen_articles` set to prevent re-processing.
  - Compiles a regular expression (`ioc_pattern`) for basic IOC detection (IPs, domains, hashes) in titles.
- **Article Fetching (`get_articles`)**:
  - Uses the `requests` library to fetch the content of `article_url`.
  - Parses the HTML response using `BeautifulSoup`.
  - Finds all <`article`> HTML elements.
  - Calls `_process_articles` to further refine the extracted data.
- **Article Processing (`_process_articles`)**:
  - Iterates through raw article elements.
  - Extracts the title and the initial redirect URL (NewsNow links are often redirects).
  - **Redirect Handling**: Attempts to follow the redirect URL to find the *actual* source article URL. It tries multiple strategies to find the real link on the redirect page (looking for specific `div` classes or any plausible outgoing link). If unsuccessful, it defaults to the redirect URL.
  - Extracts the publication timestamp.
  - Calls `_extract_iocs` to find IOCs in the article title.
  - Appends a structured dictionary (title, link, time, potential_iocs) to a list.
- **IOC Extraction (`_extract_iocs`)**:
  - Applies the pre-compiled `ioc_pattern` regex to the input text (article title).

- **Article Serialization (`serialize_article`)**:
  - Converts article data, especially `datetime` objects, into JSON-serializable formats (ISO format for time).
- **Article Analysis (`analyze_article`)**:
  - Takes an article dictionary.
  - For each `potential_ioc` found in the article, it calls `self.ioc_analyzer.generate_report(ioc)` and `self.ioc_analyzer.display_report(report)` to get the analysis.
  - Aggregates analysis results.
- **Continuous Monitoring (`monitor_and_analyze`)**:
  - Contains the main loop for the standalone script.
  - Continuously calls `get_articles`.
  - Checks if an article (based on title and link) has been seen before.
  - If new, prints article details, analyzes its IOCs using `analyze_article`, prints results, and adds the article to `seen_articles`.
  - Pauses for a specified `interval`.

2 3.2. IOC Analyzer (*ioc_analyzer.py*)

The `IOCAnalyzer` class is the analytical engine of the tool.

- **Initialization (`__init__`)**:
  - Initializes a `rich.console.Console` object.
  - Calls `_setup_gemini()` and `_setup_mitre()`.
- **Gemini AI Setup (`_setup_gemini`)**:
  - Retrieves the Gemini API key from the environment (`AI_API`).
  - Configures `genai` with the API key and specific safety settings (all categories set to `BLOCK_NONE` to ensure comprehensive analysis, assuming input is curated or risks are accepted).
  - Instantiates `genai.GenerativeModel(\"gemini-2.0-flash\", ...)`.
- **MITRE ATT&CK® Setup (`_setup_mitre`)**:
  - Defines the path for a local cache file (*cache/mitre_attack_cache.json*).
  - Creates the *cache* directory if it doesn't exist.
  - Checks if the cache file exists and is younger than `cache_age_days` (7 days). If so, loads data from it into a `stix2.MemoryStore`.
  - If the cache is missing, old, or corrupt, it downloads the latest enterprise ATT&CK JSON from MITRE's GitHub repository (*https://raw.githubusercontent.com/mitre/cti/master/enterprise-attack/enterprise-attack.json*) with a progress bar (using *tqdm*).
  - Saves the downloaded data to the cache file and loads it into the `MemoryStore`.

2.1 3.2.1. ThreatFox API Integration (*search_threatfox*)

- Queries the ThreatFox API (`https://threatfox-api.abuse.ch/api/v1/`) using a POST request with the `search_ioc` query type.
- Implements a retry mechanism (3 attempts with a 2-second delay) to handle transient network issues or API errors (like HTTP 499).
- Returns the JSON response from ThreatFox or an error structure if all retries fail.

2.2 3.2.2. MITRE ATT&CK® Framework Integration (*map_to_mitre*)

- Takes IOC data (typically from ThreatFox) as input.
- Extracts threat type, description, and malware information.
- Defines `relevant_tactics` based on common threat types (e.g., 'botnet_cc' maps to 'Initial Access', 'Command and Control').
- Queries its local `self.attack_data` (MITRE MemoryStore) for all attack patterns.
- Matches techniques if:
  - The technique's kill chain phases include any `relevant_tactics` for the IOC's threat type.
  - OR the technique's description contains keywords from the IOC's threat description or malware name.
- Returns a list of matching techniques with their ID, name, description, tactics, and URL on the MITRE website.

2.3 3.2.3. Gemini AI Analysis (*analyze_with_gemini*)

- First, calls `map_to_mitre` to get MITRE context for the IOC data.
- Constructs a detailed prompt for Gemini AI, including the IOC data (from ThreatFox) and the MITRE mapping. The prompt asks Gemini to provide:
  1. Threat Level: [Low/Medium/High]
  2. Impact: [Brief description of potential impact]
  3. Recommended Actions: [List key actions]
  4. Related Threat Actors: [If any]
  5. Historical Context: [If available]
- Sends the prompt to the configured Gemini model.
- Parses Gemini's text response, expecting the structured format requested in the prompt, and populates a dictionary with these sections.
- Returns a dictionary containing the parsed AI analysis, the raw IOC data, and the MITRE mapping.

2.4 3.2.4. Report Generation (*generate_report*, *display_report*)

- **generate_report(ioc)**:
  - Orchestrates IOC analysis: calls `search_threatfox(ioc)` and then `analyze_with_gemini()` with the ThreatFox data.

  - Returns a consolidated dictionary with the original IOC, ThreatFox data, and the full AI analysis object.

- **display_report(report)**:
  - Takes the report dictionary generated by `generate_report`.
  - Formats the entire report into an HTML string. This includes:
    * Basic IOC information.
    * AI Analysis sections (Threat Level, Impact, etc.).
    * Detailed ThreatFox data (using `rich.table.Table` converted to HTML, or direct HTML formatting).
    * MITRE ATT&CK® Mappings (also potentially using `rich.table.Table` or direct HTML).
  - Wraps the output in a `<div class='report-container'>`.

2.5 3.2.5. Article Content Analysis (*analyze_article_content*, *_format_article_analysis_html*)

- **`analyze_article_content(content)`**:
  - Takes raw article text as input.
  - Constructs a prompt for Gemini AI to:
  1. Summarize the article.
  2. List key entities (organizations, malware, vulnerabilities).
  3. Assess the overall threat.
  - Parses Gemini's response into sections.
- **`_format_article_analysis_html(sections)`**:
  - Formats the parsed article analysis into an HTML string.

3 3.3. Web Interface and API (*web_monitor.py*)

This module uses FastAPI to create the web application and its associated APIs.

- **Application Setup**:
  - Initializes a `FastAPI` app instance.
  - Uses an `asynccontextmanager` named `lifespan` to initialize `CyberMonitor` and `IOCAnalyzer` instances on application startup and start the `monitor_thread` background task.
  - Mounts a `/static` directory for static assets (CSS, JS, images).
  - Configures `Jinja2Templates` for rendering HTML pages from a "templates" directory.
- **Background Monitoring (`monitor_thread`)**:
  - Runs as an `asyncio` task.
  - Includes a demo article that is always present in the `recent_articles` list.

  - Periodically (every 10 minutes / 600 seconds in the current code, although this can be adjusted as needed) calls `monitor.get_articles()`.
  - For new articles, it calls `monitor.analyze_article()`.
  - Appends the new article and its analysis to a global `recent_articles` list (capped at 50 new + 1 demo).
  - Broadcasts the new article data to all connected WebSocket clients using `broadcast_message`.

- **WebSocket Communication (`websocket_endpoint`, `broadcast_message`)**:
  - **`/ws` endpoint**: Handles WebSocket connections.
  - On connection, adds the client to an `active_connections` set.
  - Sends the current `recent_articles` list to the newly connected client (`initial_data`).
  - Listens for messages; primarily for keep-alive or future client-to-server commands (currently sends "pong").
  - Removes clients from `active_connections` on disconnect or error.
  - **`broadcast_message(message)`**: Iterates through `active_connections` and sends JSON messages. It ensures `datetime` objects are serialized before sending.

- **Main Web Page (`/`)**:
  - Renders `templates/monitor.html`, passing the current `recent_articles` to the template for display.

- **Key API Endpoints**:
  - **`GET /recent_iocs`**:
  - Queries the ThreatFox API for IOCs reported in the last day (`query: get_iocs, days: 1`).
  - Returns the latest 10 IOCs as JSON. Includes error handling for API issues.
  - **`POST /analyze`**:
  - Expects an `ioc` in the request payload.
  - Calls `ioc_analyzer.generate_report(ioc)` and `ioc_analyzer.display_report(report)` to get the HTML analysis.
  - Calls `check_pcap_for_ioc(ioc)` to scan a local `.pcapng` file.
  - Returns a JSON response containing the `html_report` and `pcap_check_result`.
  - **`POST /analyze_article`**:
  - Expects `article_content` in the request payload.
  - Calls `ioc_analyzer.analyze_article_content(article_content)` and `ioc_analyzer._format_article_analysis_html()`.
  - Returns a JSON response with the `html_analysis`.

- **PCAP Analysis (`check_pcap_for_ioc`)**:
  - Searches for the first `.pcapng` file in the application's root directory.
  - Uses `subprocess.run` to execute `tshark -r <pcap_file> -Y 'frame contains \ "<ioc_search_term>\"'`.
  - (Note: The IOC is slightly modified by removing "www." before searching).
  - Returns whether the IOC was found and any matching packet details from `tshark`'s output.

4 3.4. RSS Feed Monitor (*rss_monitor.py*)

The `rss_monitor.py` script is a more straightforward component for fetching data from RSS feeds.

- **Functionality**:
  - Uses the `feedparser` library.
  - Is configured with the ThreatPost RSS feed URL (`https://threatpost.com/feed/`).
  - Parses the feed.
  - Prints the feed title and the title of the most recent article.
- **Current State**: As implemented, it's a one-shot script. It does not have a continuous monitoring loop or directly integrate into the `WebMonitor`'s background task or `CyberMonitor`'s data flow for automated, ongoing analysis in the main application. It serves as a proof-of-concept or utility for fetching from ThreatPost. To meet FR01 fully for ThreatPost in an automated way similar to NewsNow, its logic would need to be integrated into a persistent monitoring loop, likely within the `CyberMonitor` class or the `web_monitor.py` background task.

CHAPTER

FOUR

4. USER GUIDE

This guide provides instructions on how to set up, run, and use the AI-Powered Cyber Incident Monitoring Tool.

1 4.1. Installation and Setup

1.1 4.1.1. Prerequisites

Before installing the tool, ensure you have the following prerequisites:

- **Python**: Version 3.8 or higher.
- **pip**: Python package installer (usually comes with Python).
- **Git**: For cloning the repository.
- **tshark**: The command-line utility for Wireshark. This must be installed and accessible in your system's PATH for PCAP analysis functionality. Installation varies by OS:
  - **Linux (Debian/Ubuntu)**: `sudo apt update && sudo apt install tshark`
  - **Linux (Fedora)**: `sudo dnf install wireshark-cli`
  - **macOS (using Homebrew)**: `brew install wireshark` (tshark is included)
  - **Windows**: Install Wireshark from the official website and ensure the installation directory (containing tshark.exe) is added to your system's PATH environment variable.
- **Internet Access**: Required for fetching articles, accessing ThreatFox API, and Google Gemini AI API.

1.2 4.1.2. Installation Steps

1. **Clone the Repository**:

   Open your terminal or command prompt and navigate to the directory where you want to install the project. Then run:

   ```
   git clone https://github.com/GGal1leo/CYB_FYP/
   cd CYB_FYP # Navigate into the cloned project directory
   ```

2. **Create a Virtual Environment** (Recommended):

   It's highly recommended to use a virtual environment to manage project dependencies.

```
python -m venv venv
```

3. **Activate the Virtual Environment**:
   - **Linux/macOS**:

```
source venv/bin/activate
```

   - **Windows (Command Prompt)**:

```
venv\\Scripts\\activate.bat
```

   - **Windows (PowerShell)**:

```
.\\venv\\Scripts\\Activate.ps1
```

4. **Install Dependencies**:

   With the virtual environment activated, install the required Python packages:

```
pip install -r requirements.txt
```

5. **Configure Environment Variables**:

   The tool requires API keys for Google Gemini AI. These are managed using a `.env` file.

   - Copy the example environment file:

```
cp .env.example .env
```

   - Edit the `.env` file with your actual API key:

```
AI_API="YOUR_GEMINI_API_KEY_HERE"
```

     Replace `YOUR_GEMINI_API_KEY_HERE` with your valid Google Gemini API key.

2 4.2. Running the Application

2.1 4.2.1. Web Interface

To start the web interface and background monitoring tasks:

1. Ensure your virtual environment is activated.
2. Navigate to the project's root directory in your terminal.
3. Run the Uvicorn server:

```
uvicorn web_monitor:app --reload
```

   - `web_monitor:app` tells Uvicorn to find the FastAPI application instance named `app` within the `web_monitor.py` file.
   - `--reload` enables auto-reloading, so the server will restart if you make changes to the Python code (useful for development).

4. Once the server is running (it will typically say "Application startup complete" and indicate it's listening on `http://127.0.0.1:8000`), open your web browser and navigate to: `http://127.0.0.1:8000`

2.2 4.2.2. Command-Line Monitoring (*cyber_monitor.py*)

The `cyber_monitor.py` script can be run directly to perform continuous monitoring and print results to the console (as seen in its `if __name__ == "__main__":` block). This is primarily for development or a headless monitoring setup.

1. Ensure your virtual environment is activated and `.env` is configured.
2. Run:

```
python cyber_monitor.py
```

This will start fetching articles from NewsNow and analyzing them, printing output to the terminal. Note that this runs independently of the web interface's background task unless its internal logic is modified.

2.3 4.2.3. RSS Feed Fetching (*rss_monitor.py*)

The `rss_monitor.py` script is a simple utility to fetch the latest from the ThreatPost RSS feed.

1. Ensure your virtual environment is activated.
2. Run:

```
python rss_monitor.py
```

This will print the latest article title from ThreatPost to the console.

3 4.3. Using the Web Interface

The web interface provides a central dashboard for monitoring and analyzing cyber threats.

- **Dashboard**:
  - Displays a list of recently monitored articles from NewsNow. New articles appear in real-time at the bottom of the list.
  - Each article entry shows the title (clickable), publication time, and any initially extracted IOCs.
  - A "Demo Article" is always present for demonstration purposes.
- **Viewing Article Analysis**:
  - Clicking on an article title will typically expand or show the AI-generated analysis for that article (if this functionality is fully implemented in the `monitor.html` template to display `analysis['ioc_analysis']` for each article).
- **Manual IOC Analysis**:
  - Find the "Analyze IOC" section.
  - Enter an IOC (e.g., an IP address, domain, URL, or hash) into the input field.

  - Click “Analyze”. The system will query ThreatFox, use Gemini AI for analysis, map to MITRE ATT&CK®, and check against a local PCAP file.
  - The detailed HTML report and PCAP scan results will be displayed on the page.

- **Manual Article Content Analysis**:
  - Find the “Analyze Article Content” section.
  - Paste the full text of a news article or threat report into the text area.
  - Click “Analyze Article”. Gemini AI will summarize the content, identify key entities, and assess the threat.
  - The HTML formatted analysis will be displayed.

- **Recent ThreatFox IOCs**:
  - A section on the page (typically “Recent IOCs from ThreatFox”) displays the last 10 IOCs reported to ThreatFox in the past day.

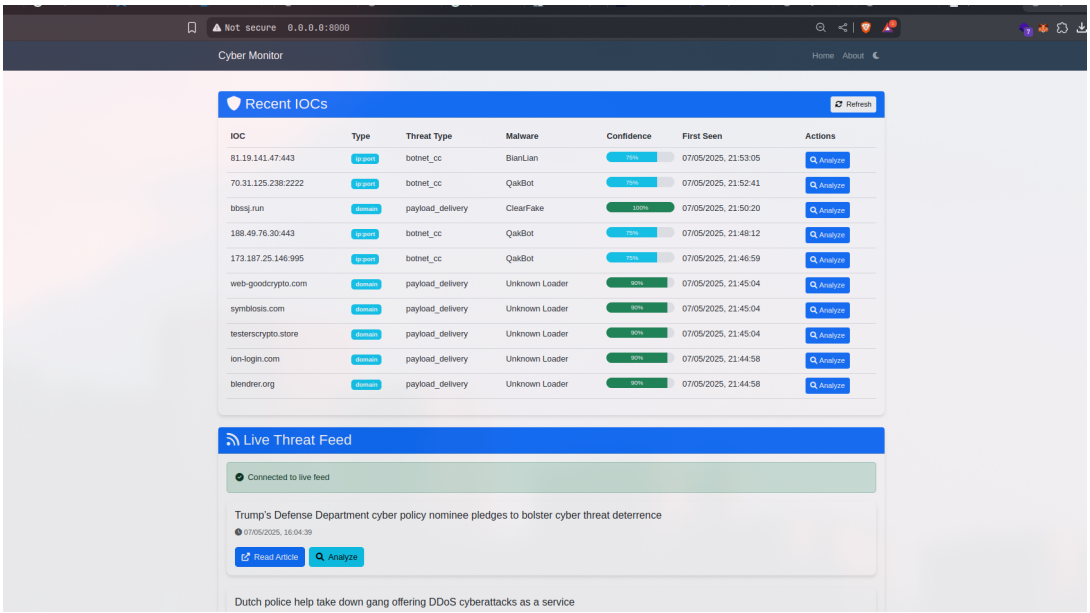

Fig. 1: Layout of the web interface dashboard, showing recent articles and analysis sections.

4 4.4. Using the Command-Line Interface (CLI) for IOC Analysis

Functional Requirement FR15 specifies a CLI for submitting an IOC for analysis and receiving a report. While `cyber_monitor.py` and `rss_monitor.py` have command-line execution modes for their specific tasks, a dedicated CLI for on-demand IOC analysis as per FR15 would typically involve:

1. A script (e.g., `cli_analyzer.py`, or an `if __name__ == "__main__":` block in `ioc_analyzer.py`) that accepts an IOC as a command-line argument.
2. This script would then use the `IOCAnalyzer` class to:

- Call `generate_report(ioc_argument)`.
- Either print a textual summary of the report to the console or save the HTML report (from `display_report()`) to a file and print the file path.

Example (Conceptual Usage): Assuming such a CLI script (e.g., `cli_analyzer.py`) is created:

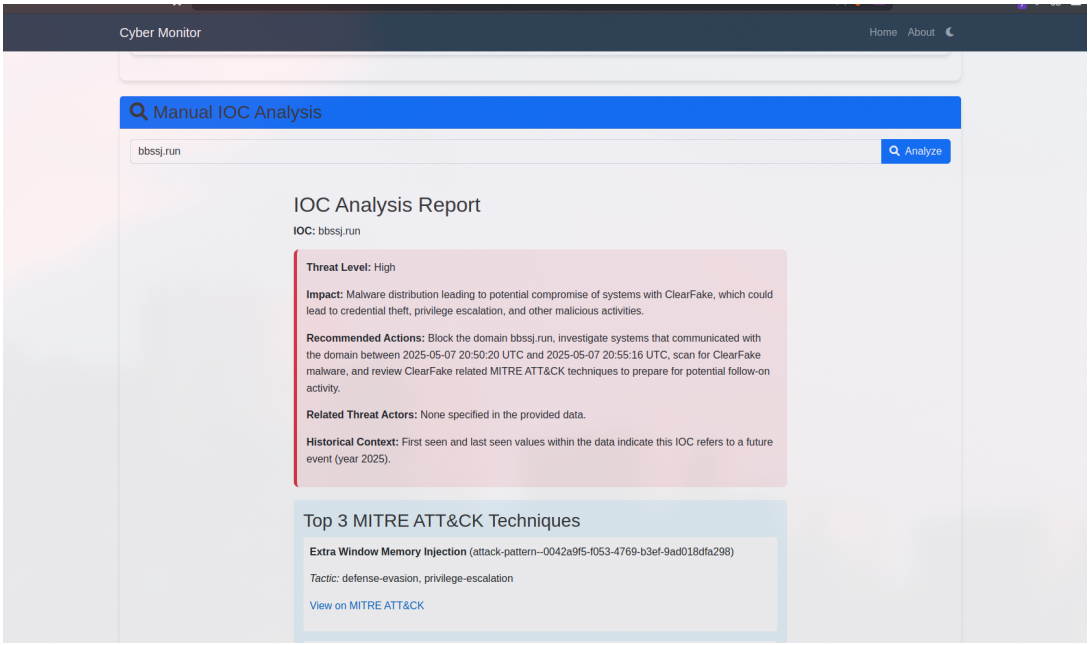

Fig. 2: Example of how a detailed IOC analysis report might be displayed in the web interface.

```
python cli_analyzer.py --ioc "1.2.3.4"
# Or:
python cli_analyzer.py --ioc "evil-domain.com" --output-html report_for_evil_
↪domain.html
```

Currently, the provided codebase focuses `ioc_analyzer.py` as a library. To fully meet FR15, a small wrapper script or main execution block in `ioc_analyzer.py` would be beneficial to provide this direct CLI IOC analysis capability.

CHAPTER

FIVE

5. API DOCUMENTATION

This section provides an overview of the main programmatic interfaces (classes and key methods) within the project. For detailed type hints and internal logic, please refer to the source code comments and Python type annotations.

1 5.1. *IOCAnalyzer* Class (*ioc_analyzer.py*)

The central class for all IOC and article analysis tasks.

- **`__init__(self)`**:
  - Initializes Gemini AI, MITRE ATT&CK® cache, and console utilities.
- **`search_threatfox(self, ioc: str) -> Dict`**:
  - Queries ThreatFox API for information about the given `ioc`.
  - Returns a dictionary with ThreatFox data.
- **`map_to_mitre(self, ioc_data: Dict) -> List[Dict]`**:
  - Maps enriched `ioc_data` (usually from ThreatFox) to MITRE ATT&CK® techniques.
  - Returns a list of dictionaries, each representing a matched technique.
- **`analyze_with_gemini(self, ioc_data: Dict) -> Dict`**:
  - Sends `ioc_data` (including MITRE mappings) to Gemini AI for detailed analysis (threat level, impact, recommendations).
  - Returns a dictionary with the structured AI analysis.
- **`generate_report(self, ioc: str) -> Dict`**:
  - Orchestrates the full analysis of an `ioc` by calling `search_threatfox` and `analyze_with_gemini`.
  - Returns a comprehensive report dictionary.
- **`display_report(self, report: Dict) -> str`**:
  - Formats the comprehensive `report` dictionary into an HTML string for display.
- **`analyze_article_content(self, content: str) -> Dict`**:
  - Sends full `content` text to Gemini AI for summarization and threat assessment.
  - Returns a dictionary of the parsed AI analysis sections.
- **`_format_article_analysis_html(self, sections: Dict) -> str`**:

  - Formats the article analysis `sections` into an HTML string.

2 5.2. *CyberMonitor* Class (*cyber_monitor.py*)

Manages fetching and initial processing of articles from web sources.

- **`__init__(self)`**:
  - Initializes an `IOCAnalyzer` instance, sets target URL, headers, and IOC patterns.
- **`get_articles(self) -> List[Dict]`**:
  - Fetches and parses articles from the configured URL.
  - Returns a list of processed article dictionaries.
- **`analyze_article(self, article: Dict) -> Dict`**:
  - Analyzes IOCs found in a given `article` dictionary using its `IOCAnalyzer` instance.
  - Returns analysis results.
- **`monitor_and_analyze(self, interval: int = 60)`**:
  - The main loop for standalone execution, continuously fetching and analyzing new articles.
- **`serialize_article(self, article: Dict) -> Dict`**:
  - Converts an article dictionary to a JSON-serializable format.

3 5.3. Web API Endpoints (*web_monitor.py*)

Key FastAPI endpoints for web interface interaction and AJAX calls.

- **`GET /`**:
  - Serves the main HTML page (*monitor.html*).
- **`WEBSOCKET /ws`**:
  - Handles WebSocket connections for real-time updates to clients.
  - Sends initial data and broadcasts new article analyses.
- **`GET /recent_iocs`**:
  - Fetches and returns the last 10 IOCs from ThreatFox (reported in the last day) as JSON.
- **`POST /analyze`**:
  - **Request**: JSON payload with `{"ioc": "ioc_value_here"}`.
  - **Response**: JSON with `{"html_report": "...", "pcap_check_result": "..."}`.
  - Analyzes the provided IOC using `IOCAnalyzer` and checks it against a local PCAP file.
- **`POST /analyze_article`**:
  - **Request**: JSON payload with `{"article_content": "full_text_here"}`.
  - **Response**: JSON with `{"html_analysis": "..."}`.
  - Analyzes the provided article text using `IOCAnalyzer`.

CHAPTER

SIX

6. SECURITY CONSIDERATIONS

A thorough examination of security implications is crucial for any tool handling threat intelligence and interacting with external services.

1 6.1. API Key Management

- **Issue**: The tool relies on API keys for Google Gemini AI. If these keys are hardcoded or mismanaged, they can be compromised, leading to unauthorized API usage and potential costs or service disruption.
- **Mitigation**: API keys are rightly managed using a `.env` file and loaded via `python-dotenv`. The `.env` file should be included in `.gitignore` to prevent accidental commitment to version control. Users must be instructed on securely creating and populating this file.
- **Industry Context**: Secure API key management is a fundamental security practice. Breaches often occur due to exposed credentials in code repositories.

2 6.2. Input Validation and Sanitization

- **Issue**: The system accepts user input for IOCs (web UI, CLI) and article content. It also processes data from external web pages and APIs. Maliciously crafted inputs could potentially lead to injection attacks (e.g., if inputs were directly used in OS commands without care, though `subprocess.run` with a list of arguments is safer) or cause unexpected behavior in parsing and analysis logic.
- **Mitigation**:
  - **IOCs**: While specific validation patterns for IOCs are inherently part of the analysis, inputs should be treated as untrusted. When displaying IOCs or data derived from them (e.g., in HTML reports), ensure proper output encoding/escaping if not already handled by frameworks like Jinja2 to prevent XSS.
  - **Article Content**: Text sent to Gemini AI should be considered. While Gemini has its own safety filters, large or malformed inputs could impact performance or lead to errors.
  - **``tshark`` Interaction**: The `check_pcap_for_ioc` function in `web_monitor.py` constructs a command for `tshark`. Although it uses `subprocess.run` with a list of arguments (which is good practice against shell injection), the IOC itself is embedded in a filter string. While `tshark`'s filter syntax is specific, extreme IOCs with special characters could theoretically cause issues if not handled or quoted perfectly by `tshark` itself. The current implementation where `www.` is removed is a slight modification but doesn't represent full sanitization.
- **Industry Context**: Input validation is a cornerstone of web application security (OWASP Top 10). Lack of it leads to various vulnerabilities.

3 6.3. External API Interactions

- **Issue**: The tool depends on external APIs (Gemini, ThreatFox). These services could be unavailable, be compromised, or return malicious/unexpected data.
- **Mitigation**:
  - **HTTPS**: All API calls use HTTPS, which is essential.
  - **Error Handling**: The code includes retries and error handling for API calls (e.g., in `IOCAnalyzer.search_threatfox`), which is good for reliability.
  - **Data Trust**: Data from external APIs, while valuable, should not be implicitly trusted for critical actions without further validation or contextualization where possible. The AI analysis step helps add a layer of interpretation.
- **Industry Context**: Supply chain security, including the security of third-party APIs, is a growing concern. Organizations must be aware of the risks associated with external dependencies.

4 6.4. Web Application Security (FastAPI)

- **Issue**: As a web application, it's exposed to common web vulnerabilities if not carefully developed.
- **Mitigation**:
  - **FastAPI Defaults**: FastAPI provides some built-in protections (e.g., data validation via Pydantic, automatic OpenAPI documentation which helps in understanding an API surface).
  - **Output Encoding**: Jinja2, used for templating, generally auto-escapes data, which helps prevent Cross-Site Scripting (XSS). This should be relied upon and understood.
  - **CSRF**: Since the app involves POST requests that perform actions (analyze IOC/article), Cross-Site Request Forgery (CSRF) could be a concern if user sessions/authentication were more advanced. For the current scope (no advanced auth), the risk is lower but worth noting for future enhancements.
  - **Dependency Security**: Keep FastAPI and other dependencies updated to patch known vulnerabilities.
- **Industry Context**: OWASP Top 10 provides a list of critical web application security risks.

5 6.5. Local File System Interaction

- **Issue**: The tool interacts with the local file system for PCAP file analysis and MITRE cache.
- **Mitigation**:
  - **PCAP Path**: `check_pcap_for_ioc` looks for the *first* `.pcapng` file in the root directory. This is predictable but could be an issue if multiple unrelated PCAP files exist or if the application's root directory is unexpectedly broad. A more explicit path configuration or selection mechanism would be safer in a multi-user or complex environment.
  - **``tshark`` Execution**: As mentioned, using `subprocess.run` with a command list is good. Direct shell execution (`shell=True`) should be avoided.
  - **Cache Directory**: The `ioc_analyzer.py` creates a `cache` directory. Ensure appropriate permissions if the tool were to run in a shared environment.

- **Industry Context**: Path traversal and insecure file system operations can lead to unauthorized file access or execution.

6 6.6. Data Privacy

- **Issue**: The tool processes potentially sensitive information (IOCs, article content which might contain internal details if a user analyzes private reports, PCAP file data).
- **Mitigation**:
  - **IOCs/Articles**: Data sent to Gemini AI and ThreatFox is subject to their respective privacy policies. Users should be aware of this.
  - **PCAP Files**: These can contain highly sensitive network traffic. The tool processes them locally with `tshark`. No PCAP data is transmitted externally by this tool itself.
  - **Data at Rest**: The MITRE cache is public data. API keys are in `.env`. No other long-term storage of processed IOCs or reports is explicitly implemented in a database, reducing persistent data privacy risks within the tool itself (beyond in-memory `recent_articles`).
- **Industry Context**: Data privacy regulations (like GDPR) are strict. Tools handling potentially sensitive data must consider data minimization, purpose limitation, and user consent.

7 6.7. Broader Industry Context and Relevance

This tool directly addresses several current industry challenges:

- **Threat Intelligence Overload**: By automating collection and initial analysis, it helps security teams cope with the vast amount of available threat data.
- **Need for Contextualization**: Integrating ThreatFox and MITRE ATT&CK® provides crucial context that raw IOCs often lack, enabling more informed decision-making.
- **AI in Cybersecurity**: It demonstrates a practical application of Large Language Models (LLMs) like Gemini to augment cybersecurity analysis, a rapidly growing trend.
- **Improving Incident Response Time**: Faster identification and analysis of relevant IOCs can significantly shorten the incident response lifecycle.
- **Skill Augmentation**: The tool can assist less experienced analysts by providing structured analysis and recommendations.

However, reliance on such tools also highlights the importance of human oversight. AI analysis, while powerful, is not infallible and should be reviewed by experienced professionals.

CHAPTER

SEVEN

7. PROJECT REFLECTION

This section reflects on the development process of the AI-Powered Cyber Incident Monitoring Tool, covering achievements, challenges, lessons learned, and potential future directions, in line with the project's grading criteria.

1 7.1. Achievements and What Was Not Achieved

1.1 7.1.1. Key Achievements

The project successfully delivered a functional prototype demonstrating the core vision of an AI-augmented cyber incident monitoring tool.

- **Automated Article Ingestion & IOC Extraction (FR01, FR02)**: The `CyberMonitor` module successfully fetches articles from NewsNow and extracts potential IOCs from titles using regex.
- **AI-Powered IOC Analysis (FR04)**: The `IOCAnalyzer` effectively uses the Gemini AI API to perform detailed analysis of IOCs, generating structured summaries covering threat level, impact, and recommended actions.
- **ThreatFox Integration (FR03)**: IOCs are successfully enriched with data from the ThreatFox API, providing valuable external intelligence.
- **MITRE ATT&CK® Mapping (FR06)**: Enriched IOC data is mapped to relevant MITRE ATT&CK® techniques using a local cache, providing standardized threat context.
- **Web Interface with Real-Time Updates (FR08, FR09, FR11, FR12)**: The FastAPI web application (`web_monitor.py`) provides a dashboard for recently monitored articles. It supports manual IOC submission and displays detailed HTML reports. Real-time updates for new articles are handled via WebSockets.
- **Manual IOC & Article Analysis via Web UI (FR10, FR05)**: Users can manually submit IOCs for full analysis or paste article content for AI-driven summarization and analysis through the web interface.
- **PCAP File Analysis (FR13)**: The web interface allows checking for the presence of a given IOC within a provided PCAPNG file using `tshark`.
- **Automated HTML Report Generation (FR07)**: The `IOCAnalyzer` generates user-friendly HTML reports for analyzed IOCs.
- **Secure API Key Management (NFR05)**: API keys are managed via `.env` files.
- **Modular Design (NFR04)**: The codebase is organized into distinct modules (`CyberMonitor`, `IOCAnalyzer`, `WebMonitor`), facilitating maintainability.

- **Graceful Error Handling for APIs (NFR06)**: External API call failures are handled with retries and error messages.

1.2 7.1.2. Areas Not Fully Achieved or Out of Scope

While the core objectives were met, some aspects were explicitly out of scope for this version, or represent areas for further development:

- **Sophisticated Threat Ranking (Out of Scope)**: The project specification explicitly excluded "Sophisticated, customizable threat ranking based on detailed organizational profiles." While the `Research Document` mentioned this as an objective, it was not implemented as per the formal spec. The current AI analysis provides a "Threat Level," but it's a general assessment, not tailored to a specific organizational risk profile.
- **ThreatPost RSS Integration into Main Loop (Partially Met - FR01)**: While `rss_monitor.py` can fetch from ThreatPost, it's a standalone script. Its functionality is not integrated into the continuous monitoring loop of `CyberMonitor` or `WebMonitor` for automated, ongoing analysis in the main application dashboard. This would require further integration.
- **Dedicated CLI for IOC Analysis (FR15 - Partially Met/Requires Wrapper)**: The project specification (FR15) calls for a CLI to submit an IOC for analysis and receive a report. While `ioc_analyzer.py` contains all the necessary logic, it's primarily a library. A simple wrapper script or a `if __name__ == "__main__":` block would be needed in `ioc_analyzer.py` to make it directly executable as a CLI tool for on-demand IOC analysis. `cyber_monitor.py` has a CLI execution mode, but for its own continuous monitoring task, not for on-demand IOC analysis.
- **Advanced User Authentication & RBAC (Out of Scope)**: As per the spec, these were not implemented.
- **Extensive Data Source Support (Out of Scope)**: The tool focuses on NewsNow and conceptually ThreatPost.
- **Active Remediation (Out of Scope)**: The tool is for monitoring and analysis, not active defense.

2 7.2. Problems Encountered and Solutions

The development process involved several common challenges:

- **Problem: Dynamic Web Content and Anti-Scraping**:
  - **Issue**: Some initial news sources considered were difficult to scrape reliably due to dynamically loaded JavaScript content or anti-scraping measures. NewsNow's redirect mechanism also required careful handling.
  - **Solution**: Focused on more stable sources with clearer HTML structures or RSS feeds. For NewsNow, implemented multi-step fetching to resolve redirects and employed robust parsing with `BeautifulSoup`, trying various selectors to find the true destination URL.
- **Problem: IOC Extraction Accuracy**:
  - **Issue**: Initial regex-based IOC extraction from titles was prone to false positives and missed some IOCs.
  - **Solution**: Iteratively refined regex patterns. More significantly, leveraged Gemini AI not just for analyzing confirmed IOCs, but also as part of the conceptual pipeline for *identifying* IOCs from broader text (as per FR02). The current `cyber_monitor.py` uses regex on titles; a future step could send full article text to Gemini for more nuanced IOC extraction.

- **Problem: External API Reliability and Rate Limiting**:
  - **Issue**: Calls to ThreatFox and Gemini AI APIs occasionally failed due to transient network issues or hitting rate limits during intensive testing.
  - **Solution**: Implemented retry logic with exponential backoff for ThreatFox calls in `IOCAnalyzer`. For Gemini, ensuring efficient prompting and avoiding unnecessary calls is key. Caching API responses (especially for frequently requested, non-time-sensitive data, though not explicitly implemented for ThreatFox/Gemini beyond the MITRE cache) could be a further improvement.
- **Problem: Real-time Web Interface Updates**:
  - **Issue**: Ensuring smooth and reliable real-time updates on the web dashboard via WebSockets required careful management of asynchronous tasks and client connections. Serializing complex objects (like `datetime`) for JSON also needed attention.
  - **Solution**: Used FastAPI's robust WebSocket support. Implemented a centralized `broadcast_message` function to manage sending updates to all active connections. Ensured data (like `article` objects) was serialized correctly (e.g., `monitor.serialize_article()`) before sending over WebSockets.
- **Problem: ``tshark`` Integration and Environment Dependency**:
  - **Issue**: PCAP analysis relies on `tshark` being installed and in the system PATH. This external dependency can make setup more complex for users. Capturing and parsing `tshark` output also requires care.
  - **Solution**: Clearly documented `tshark` as a prerequisite. Used `subprocess.run` to call `tshark`, which is safer than `os.system`. Parsed its output to determine if an IOC was found. More robust error handling around `tshark` execution (e.g., if it's not found) could be added.

3 7.3. Lessons Learned

This project offered significant learning experiences:

- **The Power and Challenges of LLMs**: Integrating Gemini AI demonstrated its impressive capability for text analysis, summarization, and structured data generation. However, it also highlighted the importance of clear, precise prompting and the need to parse potentially variable text outputs reliably.
- **Importance of Modular Design**: The separation of concerns into `CyberMonitor` (collection), `IOCAnalyzer` (analysis), and `WebMonitor` (presentation) was crucial for managing complexity and enabling parallel development and testing.
- **Iterative Refinement**: For features like IOC extraction, MITRE mapping, and AI prompting, an iterative approach of testing, evaluating results, and refining logic was essential to achieve acceptable accuracy and utility.
- **Handling External Dependencies**: Reliance on external APIs and tools (ThreatFox, Gemini, tshark) necessitates robust error handling, retry mechanisms, and clear documentation of these dependencies for users.
- **Asynchronous Programming**: Developing the real-time web interface with FastAPI and WebSockets provided valuable experience in asynchronous programming with `asyncio`.

- **Value of Standard Frameworks**: Using MITRE ATT&CK® provides a common language for describing threats and significantly enhances the value of the analysis by placing it within a recognized industry framework.
- **Security from the Start**: Even for a prototype, thinking about security aspects like API key management and potential input issues (NFR05) is vital.

4 7.4. What Would Be Done Differently

If starting the project over, the following aspects might be approached differently:

- **More Comprehensive Upfront API Design**: Before deep implementation, define more detailed internal API contracts between the modules (e.g., the exact data structures passed between `CyberMonitor` and `IOCAnalyzer`, and to the `WebMonitor` templates/WebSockets). This could reduce some integration friction.
- **Test-Driven Development (TDD) for Core Logic**: For critical components like IOC parsing, ThreatFox interaction, and MITRE mapping, applying TDD more rigorously from the outset could have caught edge cases earlier and ensured higher reliability.
- **Centralized Configuration Management**: While `.env` is good for API keys, a more structured configuration object or file for parameters like cache age, API URLs (if they were to change), and monitoring intervals could be beneficial.
- **Dedicated CLI Module**: Create a dedicated CLI script (e.g., *cli.py* using a library like *Typer* or *Click*) early on to provide the IOC analysis functionality (FR15) rather than relying on *if __name__ == "__main__"* blocks in library-like modules. This would make the CLI a more first-class citizen.
- **State Management for Web UI**: For more complex web interfaces, consider a frontend JavaScript framework or more sophisticated state management on the client-side rather than relying solely on full data refreshes or simple WebSocket appends via Jinja2. (Though for the current scope, the existing approach is likely adequate).
- **Database for Persistence**: For a production-grade tool, storing seen articles, analysis results, and user data in a proper database (e.g., SQLite, PostgreSQL) would be essential instead of in-memory lists, to ensure data persistence across application restarts and to handle larger datasets.

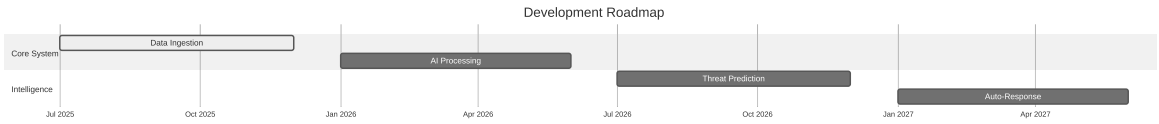

Fig. 1: Future Development Roadmap

CHAPTER

EIGHT

8. CONCLUSION

The AI-Powered Cyber Incident Monitoring Tool successfully demonstrates the feasibility and value of integrating artificial intelligence with traditional threat intelligence techniques to automate and enhance cybersecurity operations. The project achieved its primary objectives by developing a system capable of collecting threat data, extracting and enriching Indicators of Compromise using external APIs (ThreatFox) and advanced AI (Google Gemini), mapping these to the MITRE ATT&CK® framework, and presenting the findings through an accessible web interface with real-time updates and a conceptual CLI.

The tool provides a robust foundation for automated threat analysis. Key functionalities such as AI-driven IOC analysis, MITRE mapping, and PCAP IOC checking offer significant advantages over purely manual processes, enabling security teams to potentially identify and understand threats more quickly and efficiently. The modular architecture allows for future expansion and adaptation, ensuring the tool can evolve with the changing cybersecurity landscape.

While certain advanced features like organization-specific threat ranking and a fully integrated RSS monitoring loop were identified as areas for future development or were out of scope for the initial version, the core system effectively serves as a proof-of-concept and a valuable asset for cybersecurity professionals. The project underscores the transformative potential of AI in cybersecurity, while also highlighting the ongoing importance of human oversight and the need for robust security practices in the development and deployment of such tools. The insights gained during its development provide a strong basis for future enhancements that could further solidify its role in proactive cyber defense.

CHAPTER

NINE

9. REFERENCES

- **MITRE ATT&CK®**: MITRE. (2023). *MITRE ATT&CK®*. Retrieved from https://attack.mitre.org/
- **ThreatFox API**: abuse.ch. (n.d.). *ThreatFox API Documentation*. Retrieved from https://threatfox.abuse.ch/api/
- **Google Gemini AI**: Google. (n.d.). *Gemini API Documentation*. Retrieved from https://ai.google.dev/docs
- **FastAPI**: FastAPI Documentation. (n.d.). Retrieved from https://fastapi.tiangolo.com/
- **Python**: Python Software Foundation. (n.d.). *Python Language Reference*. Retrieved from https://www.python.org/
- **Beautiful Soup**: Richardson, L. (n.d.). *Beautiful Soup Documentation*. Retrieved from https://www.crummy.com/software/BeautifulSoup/bs4/doc/
- **Requests**: Reitz, K. (n.d.). *Requests: HTTP for Humans™*. Retrieved from https://requests.readthedocs.io/
- **feedparser**: feedparser documentation. (n.d.). Retrieved from https://pythonhosted.org/feedparser/
- **STIX2**: OASIS Open. (n.d.). *Structured Threat Information Expression (STIX™) Version 2.1*. Retrieved from https://docs.oasis-open.org/cti/stix/v2.1/os/stix-v2.1-os.html
- **Wireshark/tshark**: Wireshark Foundation. (n.d.). *tshark - Dump and analyze network traffic*. Retrieved from https://www.wireshark.org/docs/man-pages/tshark.html
- **Jinja2**: Jinja2 Documentation. (n.d.). Retrieved from https://jinja.palletsprojects.com/
- **Uvicorn**: Uvicorn Documentation. (n.d.). Retrieved from https://www.uvicorn.org/